

Vortex5 — Project Brief

What we are building for the Cognizant NPN SCM Hackathon, written for our mentor to review and for the team to confirm before implementation is locked. 25 September 2026. Evaluation: 28 September. Technical interview: 30 September.

1. The short version

We are building **two separate applications**, one for each use case. They do not share data, models or screens. We will demo them one after the other.

	App 1 — SupplyGuard (PR1)	App 2 — TrendWear Planner (P2)
Question it answers	We need to buy a material. Which suppliers should get how much, and will they deliver on time?	What should the factory make, when, and when should we discount it?
Main idea	Split the order across suppliers at the lowest total cost, after allowing for the risk that a supplier is late or sends bad quality.	Forecast demand, plan production against factory capacity, order fabric on time, and mark down slow sellers before the season ends.
Who it is for	Procurement team	Merchandising, production and logistics teams

Why two apps, not one:

Our mentor confirmed in writing on 25 September that P2 and PR1 are independent use cases and must have two different solutions, with no overlap in approach. Earlier verbal answers about one combined solution no longer apply.

The line between them: **SupplyGuard chooses between suppliers. TrendWear Planner decides quantities and timing**, and buys fabric from a single fixed source. That is why neither app does the other's job.

2. What each app will do

App 1 — SupplyGuard

- Hold a material requirement plan by material, plant and week, worked out from past consumption and editable by the buyer.
- Keep an approved supplier list, so only suppliers cleared for that material and plant can be chosen.
- Show a comparison table of every approved supplier: price, lead time, on-time record, defect rate and risk.
- Predict three things before we place an order: chance of late delivery, chance of quality problems, chance of the order being cancelled or only partly delivered.
- Work out the best split of an order across suppliers, respecting minimum order quantities, supplier capacity, contract limits and the delivery deadline.
- Let the user move three sliders — cost, risk and quality — and instantly see the plan change.
- Compare our plan against three simple alternatives, so we can state the saving as a number.
- Run what-if scenarios: a supplier shuts down, demand jumps 30 percent, lead times grow by two weeks, prices spike.
- Explain every recommendation in plain English.

App 2 — TrendWear Planner

- Forecast weekly demand for each clothing style.
- Forecast styles that have never been sold before, using similar past styles, which matters because TrendWear launches a new line every six weeks.
- Work out how much stock to hold as a buffer (safety stock) for each style and store, and flag anything close to a stock-out or sitting on far too much.

- Build a production plan that respects each factory's weekly capacity and flags any gap between demand and supply.
- Decide when and how much fabric to order, given minimum order sizes and a four to six week lead time.
- Track how long it takes stock to travel from each warehouse to each store, so we know the week a style actually reaches the shop floor.
- Put three plans side by side — what the buyers want to sell, what the forecast says, and what the factories can actually make — show the gap in units and in money, and agree one consensus number.
- Recommend when to start a markdown and how deep it should be, based on how fast the style is actually selling.
- Run the S&OP cycle month after month, each month re-planning the next thirteen weeks, in four stages — demand review, supply review, pre-S&OP, executive approval — saving a version of the plan at each stage.
- Give each team its own view: merchandising, production and logistics, each seeing their numbers and what they have to agree to.
- Show the money: revenue, gross margin, inventory value and distribution cost for each plan version.

3. The data

We checked the stated source of every dataset on 25 September, one by one. They are not all the same kind of data, so we sort them into **three tiers** and label every record with its tier. The app shows the tier, and **every accuracy figure we report is measured on tier 1 records only**. The mentor allows Kaggle data mixed with generated data, so all three tiers are permitted — but we describe each one honestly rather than calling everything real.

Tier	Meaning	Which datasets
1. Real world	Genuine records from a real organisation	SCMS delivery history (confirmed from USAID's own site). The procurement KPI file states it is real and anonymised.
2. Public synthetic	Published on Kaggle but generated by its author	Retail clothing sales, supplier order lines, the SAP tables, the 100-row supply chain file.
3. Generated by us	Created by our own script from the patterns in tiers 1 and 2	Extra suppliers, extra styles, warehouse lanes, capacities, minimum order sizes, the merchandising plan.

App 1 data

Source	Tier / size	What we use it for
SCMS delivery history (USAID health supply chain)	Tier 1 — 10,324 real purchase orders	Our main dataset. 73 vendors across 88 factories = 217 suppliers, 141 of them competing for the same product group. Each order has a promised date and an actual delivery date, so 11.5% are genuinely late. Also has prices, freight cost, weight, shipping mode, 43 countries.
SAP tables	Tier 2 — 2,590 suppliers; 158,942 order lines	Correction: this is Google Cloud's public demonstration data, not a real company's system export. The table structures are genuine SAP, the contents are simulated. We use it for the wider supplier catalogue and to show we can read ERP-shaped data. We never quote its numbers as real supplier behaviour.
Supplier order lines	Tier 2 — 517 orders, 44 suppliers	Quality problems: 13.7% of these orders had a material fault, with rework cost and who was at fault. The file states it is synthetic, so we treat these results as a demonstration, not evidence.
Procurement KPI file	Tier 1 — 777 orders	Disruption: 63 cancelled and 73 partly delivered orders, plus defective units and negotiated prices.

Generated suppliers	Tier 3 — about 300 added	Brings the total to roughly 520 suppliers so the allocation problem is realistically large. Their prices, lead times and reliability are copied from the patterns in the real data.
---------------------	--------------------------	---

One thing we will say out loud on stage: the use case describes a manufacturer buying raw materials, and our data is health-commodity procurement. The structure is the same — a buyer, approved suppliers, materials, plants, orders, promised and actual deliveries — so we map item to material, vendor and factory to supplier, and delivery location to plant. We explain the mapping rather than hide it behind renamed columns.

App 2 data

Source	Tier / size	What we use it for
Retail store inventory and demand data	Tier 2 — 14,626 rows of clothing sales	20 styles across 5 stores, daily for two years, with price, discount from 0 to 20 percent, promotions, seasons, weather and competitor prices. The discount column gives us the relationship between price cuts and extra sales, which is what the markdown advice is built on. The file describes itself as synthetic but realistic, so our forecast accuracy shows the method works, not that we predicted real shoppers.
Generated styles	Tier 3 — about 40 added	Extends the catalogue to roughly 60 styles arranged on a six-week launch calendar across three seasons, so the S&OP cycle and the new-style forecast have something to work on.
Generated network and plans	Tier 3 — 5 warehouses, 5 stores, 3 plants	Warehouse-to-store travel times and costs, factory weekly capacity, fabric usage per garment and minimum fabric order sizes, and the merchandising sales plan we reconcile the forecast against. All listed in an assumptions document.

Open question for the team: App 2 runs on tier 2 data. If we want genuinely real sales history there, Walmart M5 or Corporacion Favorita are real and available, at a cost of a few hours of rework. Decide before the forecasting work starts.

4. How App 2 follows the mentor's pipeline

On 25 September the mentor sent a nine-step flow for P2. Our design follows it in order, and the App 2 demo will walk through these exact steps.

Mentor's step	What we built for it
1. Dataset	Clothing sales from Kaggle, plus generated styles.
2. Data processing	Daily sales turned into weekly planning data, cleaned and checked by automated tests.
3. Demand forecasting model	LightGBM for existing styles; for brand-new styles, an average of similar past styles shaped by the usual launch curve. An open Hugging Face forecasting model is run once as a benchmark only.
4. Model evaluation	Forecast error measured against two simple baselines, on data the model never saw.
5. Inventory and safety stock	Buffer stock per style and store, worked out from how wrong our forecast usually is and how long replenishment takes, plus a reorder point and stock-out and overstock warnings.
6. S&OP planning calculation	Production plan against factory capacity, fabric ordering under minimum order sizes and lead times, and the four-stage monthly S&OP cycle.
7. P2 result	A saved plan version with the demand and supply gap, the money view, and markdown recommendations.
8. FastAPI	Every calculation is exposed as an API.
9. HTML, CSS and JS dashboard	Our React dashboard, which builds into plain HTML, CSS and JavaScript.

Step 5 was missing from our first draft and has now been added. It matters: a plan without safety stock is the first thing an experienced planner would question.

5. How App 1 follows the mentor's pipeline

The mentor also sent a flow for PR1. Our design follows it end to end.

Mentor's step	What we built for it
Supplier historical data + procurement data	Purchase orders with promised and actual dates, plus quality and cancellation records.
Pandas and NumPy, data processing	Cleaning, joining and building the features the models learn from.
Supplier risk	Three prediction models: late delivery, quality problems, disruption.
Accuracy, precision, recall, F1	All four, plus two more, measured on orders the models never saw.
Supplier comparison	One table comparing every approved supplier on price, speed, reliability, quality and risk.
PuLP, optimisation, supplier allocation	The allocation engine, built in PuLP exactly as suggested.
Cost, risk and constraint check	The objective balances cost against risk and quality; eight rules keep the answer legal and realistic.
Explainable recommendation	A reason on every line, plus a plain-English summary of the whole plan.
What-if re-run	Scenario screen: supplier outage, demand spike, longer lead times, price shock.
FastAPI, HTML/CSS/JS, PR1 dashboard	The API and the dashboard.

Worth saying on stage: the risk prediction feeds directly into the allocation. We are not picking the cheapest supplier and printing a risk report beside it — the risk changes who gets the order.

6. Checked against the official use-case document

We went through the Cognizant use-case PDF line by line on 25 September. Everything it asks for is now covered. Seven items were missing from our first draft and have been added; they are marked as added below.

What the document asks for	Where it is in our solution
PR1: allocate volumes, minimise cost, cut reliance on risky suppliers, use quality and delivery performance	The allocation engine and its comparison against three simpler ways of buying.
PR1: scenario analysis, and predict delays before the order is placed	Scenario screen; a risk check that scores a draft order before release.
PR1: material demand forecast by item and plant	Added — a requirement plan by material, plant and week.
PR1: approved supplier list	Added — only cleared suppliers can be chosen.
PR1: capacity, lead times, prices, on-time and quality scores, contract limits	All used as rules inside the allocation engine.
P2: reconcile the merchandising forecast with factory capacity and fabric lead times	Added — three plans side by side with the gap and one agreed number.
P2: a rolling monthly cycle covering sales, production and inventory	Added — the cycle now runs month after month, each re-planning thirteen weeks.

P2: markdown timing from in-season sales	The markdown planner.
P2: fabric buying against minimum order sizes and lead times	The fabric order plan.
P2: better alignment across merchandising, production and logistics	Added — one view per team, each agreeing before the plan advances.
P2: warehouse-to-store transport and lead times	Added — travel times and costs, and the week stock actually reaches each store.
Judging: architecture and the alternatives we considered	Added — a document listing every major choice and what we rejected.
Judging: real-time decisions and monitoring	Added — every result shows how long it took, and both apps have a monitoring panel.
Judging: reusability, development estimate and roadmap	Added — three short documents written at the end.

7. The models, and how we will prove they work

Model	App	How we measure it
Late delivery prediction	1	Precision, recall, F1 and ROC-AUC, compared against simply using the supplier's past late rate. Trained on older orders and tested on newer ones, never on a random split, so we cannot accidentally cheat by using the future.
Quality problem prediction	1	Same measures, on the 517-order quality file. We confirm the labels are usable before we commit to this model.
Disruption prediction	1	Same measures, on the 777-order file. We keep this model simple because the dataset is small, and we will say so. If the labels do not hold up, we ship two solid models instead of three weak ones.
Order allocation	1	Total cost against three alternatives: buy from the cheapest supplier, split equally, and copy what was historically bought. Also how much volume moves away from risky suppliers.
Demand forecast	2	WAPE and MAPE against a seasonal-naive forecast and against the forecast column already in the dataset.
New-style forecast	2	We hide one style completely and forecast it from nothing using similar styles, then score it against its real sales, alongside a category-average baseline and an open Hugging Face model run as a benchmark.
Safety stock	2	Checked against the real inventory history: how often the recommended buffer would have prevented a stock-out, and how much stock it would have tied up.
Markdown advice	2	We fit the real relationship between discount and extra sales from the data, a simple regression rather than a black box, and report margin recovered against no markdown and against a fixed end-of-season sale.

How we choose each model. For supplier risk we train logistic regression, random forest and XGBoost on the same features and the same split, show all three side by side, and keep the winner. That is ten minutes of work and it lets us answer the obvious question — why this model and not another — with a table instead of an opinion.

If a model performs worse than we hoped, we report the real number and explain it. We will be questioned on these figures on 30 September, so nothing gets rounded up.

8. Making the live demo safe

The evaluation is a live demo in a room, so we treat the demo itself as something we build and test, not something we hope goes well.

- Nothing is trained during the demo. Models are trained and checked beforehand and saved to disk; the demo only runs the finished models.
- Everything is repeatable. Random seeds and data versions are fixed, so the same scenario gives the same numbers every time.
- Each app has two scenarios ready to open: one normal, one disruption. The panel can change inputs and re-run without waiting for anything to retrain.
- Each result shows how long it took to calculate, which is also our evidence for real-time decision making.
- We test an offline version on the demo laptop the day before, and keep a recorded video as the last resort.
- A written pre-demo checklist gets run twice: on the 27th and on the morning of the 28th.

9. How this would connect to a real company

Our prototype reads exported files. It does not connect to a live ERP system, and we will say so plainly rather than imply otherwise. What we show is where a real system would attach:

Step	In a production deployment
1. Source	ERP or SAP, through a scheduled extract or an API.
2. Preparation	Validation and transformation of that data.
3. Planning engine	Exactly what we built: the models and the optimiser.
4. Approval	A planner reviews and approves the recommendation.
5. Execution	The agreed allocation or plan is written back to the ERP.

Because every calculation in our apps already sits behind an API, moving from prototype to production changes the data source and adds a write-back. It does not require a rewrite. That API layer is the integration point we will show on the architecture slide.

10. Technology

Part	What we use	Why
Screens	Plain HTML, CSS and JavaScript, with Chart.js	Team decision on 25 September: no framework. We all read plain JavaScript better than React, so anyone can explain any part of the front end on the 30th. It is also literally what the mentor's diagrams ask for. No build step, and the sortable supplier table is about 150 lines we wrote ourselves, which is easier to defend than a third-party grid.
Server	Python with FastAPI	Standard, fast, and gives automatic API documentation.
Models	LightGBM for demand, XGBoost for supplier risk	Only open-source models, as required. For every risk target we also train logistic regression and random forest on the same data, show all three, and keep the winner, preferring the simpler model when it is just as good.
Explanations	Written templates filled with our own numbers	Part of the core, because an explainable recommendation is required. An optional open language model can rewrite them more fluently, but it is off by default and the first thing we cut.
Optimisation	PuLP with the CBC solver	Straightforward to explain in an interview.
Database	SQLite	No setup needed, and easy to move to a bigger database later.

Where it runs	Two Hugging Face Spaces, one per app	Two public links, free, and GitHub Actions deploys automatically on every push.
Code	One GitHub repository, NoumanS-20/NPN	Two app folders inside, so the separation is visible.

11. Plan for the next three days

When	What gets built
25 Sep	Project skeleton, automated tests, the data pipeline for App 1, and the supplier list of about 520.
26 Sep	App 1 in full: the three risk models, the allocation engine, the scenarios and all its screens.
27 Sep	App 2 in full: forecast, production plan, fabric ordering, markdowns and the S&OP cycle. Both apps deployed. First rehearsal.
28 Sep	Final checks, second rehearsal in the morning, then the evaluation.

Rule we are sticking to, and the team agreed it: build a thin working core first — data, forecast, plan, dashboard, end to end — and only then add the advanced parts. If time runs out, we drop the plain-English explanations first, then the model monitoring, then the SAP supplier catalogue. The core of both apps is never at risk.

12. What we are deliberately not building

- No chatbot. Neither of our two use cases asks for one — that belongs to PR2, a different team's use case.
- No invoice processing, OCR or purchase-to-pay automation. Also PR2.
- No live connection to a real ERP system. We read exported SAP files instead.
- No login system or mobile app. They would cost a day and earn no marks.

13. What the team needs to do

Item	Who / by when
Decide whether App 2 moves onto genuinely real sales data	Blocks the forecasting work — needs a decision today
Find a college mentor, which is mandatory for every team	Needs an owner today
Confirm the deadline and email address for the design submission	Needs an owner today
Decide who presents which part	After the apps are built
Read the module guides we will write, so anyone can answer on any part	Everyone, by 27 Sep
Two full rehearsals	Evening of 27 Sep and morning of 28 Sep — already confirmed
Slide deck	Decided after the apps are working

14. Risks and what we do about them

Risk	What we do
The live demo fails on the day	Two public links, both apps also run offline on a laptop, and a recorded video as the last resort.
Judges think the two apps overlap	The separation rule is written in the code, the README and the demo script, and one of us states it out loud.
Someone is asked about a part they did not build	A short guide per module, plus two rehearsals where we question each other.

A model's accuracy looks weak	We explain the imbalance honestly, show the comparison against simple baselines, and say what we would do with more time.
We run out of time	A fixed cut list, agreed in advance, so we never argue about it on the last day.

15. What we would like the mentor to confirm

Item	What we propose	Feedback wanted
Risk models	XGBoost, with logistic regression and random forest as benchmarks; simpler model kept when equally good.	Is this level of simplicity right for our dataset sizes and for defending it live?
Allocation engine	PuLP with CBC; risk and quality enter the objective; minimum order sizes, capacity, contracts and deadlines as hard rules.	Is the objective, and the way we penalise risk, the right formulation?
Demand forecast	LightGBM against a seasonal-naive baseline, weekly per style, thirteen-week horizon.	Is the horizon and the feature set appropriate?
New styles	Average of similar past styles, with an open model as a benchmark.	Is the simpler method adequate for judging?
Data	Real procurement history for App 1; synthetic clothing sales for App 2.	Is synthetic sales data acceptable for P2, or should we move to real sales history?
Headline numbers	Cost saved, risky volume reduced, forecast error against baseline, margin recovered.	Which of these should lead the final presentation?
ERP story	Exported data now, with our API layer shown as the production integration point.	Is this the right way to describe the boundary?

Our own rule, agreed by the team: build a thin working core first, data to model to plan to dashboard, end to end, and only then add explanations, monitoring and secondary features. The core business decision stays stable throughout the build.

Please read this and tell Nouman if anything looks wrong, missing, or different from what you expected. Once everyone confirms, we start building.